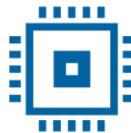




**Universitatea
Transilvania
din Braşov**



**Universitatea
Transilvania
din Braşov**
FACULTATEA DE INGINERIE ELECTRICĂ
ŞI ŞTIINŢA CALCULATOARELOR

PROIECT JAVA

Zărnescu Raul-Claudiu

4LF842 ROBO II



Sisea Ianis-Cristian

4LF842 ROBO II

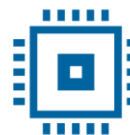


David Rareş Cosmin

4LF841 ROBO II

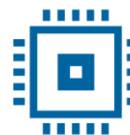


BRAŞOV, 2026



Cuprins

1. INTRODUCERE ȘI PREGĂTIREA PROIECTULUI.....	4
1.1. Obiectivele proiectului	4
1.2. Justificarea temei.....	4
1.3. Tehnologii și instrumente utilizate.....	5
1.3.1. Mediul de dezvoltare și servere:	5
1.3.2. Limbaje și tehnologii de programare:	5
1.3.3. Instrumente și biblioteci auxiliare:	5
2. STADIUL ACTUAL ȘI ANALIZA TEORETICĂ.....	5
2.1. Descrierea stadiului actual în aria temei proiectului	5
2.2. Studiul algoritmilor clasici utilizați în domeniu.....	6
2.3. Analiza comparativă a soluțiilor existente	7
3. PROIECTAREA SISTEMULUI/APLICAȚIEI	7
3.1. Arhitectura generală a sistemului.....	7
3.2. Proiectarea bazei de date / Structuri de date utilizate	8
3.3. Descrierea algoritmilor utilizați sau proiectați	9
3.3.1. Algoritm de Securitate și Criptare (Hibrid)	9
3.3.2. Algoritm de Detectare a Conflictelor (Suprapunerea Intervalelor)	9
3.3.3. Algoritmul „Debounce” (Lazy search/ Căutare leneșă)	10
3.3.4. Algoritmul „Hydration” & SPA Routing.....	10
3.3.5. Algoritm fiscal (TVA multiplu).....	11
3.3.6. Ripple Effect (efectul de undă).....	11
3.4. Diagrama claselor și a obiectelor (UML).....	13
4. IMPLEMENTAREA SOFTWARE.....	15
4.1. Descrierea modulelor software importante	15
4.1.1. Module Backend (Controllere și Core).....	15
4.1.2. Module Frontend (Client-Side).....	16
4.2. Funcțiile principale și parametrii utilizați	16
4.2.1. Funcții Backend (PHP Controllers)	16
4.2.2. Funcții Frontend (JavaScript)	17
4.3. Detalii de implementare	17
4.3.1. Arhitectura și Design Patterns	17
4.3.2. Securitate și Gestiunea Datelor.....	18
4.3.3. Ingineria Frontend și Starea Aplicației	18



4.4.	Descrierea interfeței grafice (GUI)	18
4.4.1.	Identitate vizuală și ergonomie	18
4.4.2.	Organizarea layout-urilor principale.....	19
4.4.3.	Elemente interactive și optimizări vizuale.....	20
5.	TESTARE ȘI REZULTATE	21
5.1.	Scenarii de testare	21
A.	Scenariul: Comandă la Masă (Validare prin Token QR).....	21
B.	Scenariul: Plată Securizată (Criptare Hibridă).....	23
C.	Scenariul: Rezervare cu Suprapunere (Logică de Business).....	23
5.2.	Analiza performanțelor aplicației.....	24
A.	Optimizarea paginii principale (Homepage).....	24
B.	Eficientizarea navigării în modulul de meniu	24
6.	CONCLUZII.....	26
6.1.	Concluzii generale.....	26
6.2.	Elemente de originalitate.....	26
6.3.	Direcții viitoare de dezvoltare	26
6.4.	Impresii	26
7.	BIBLIOGRAFIE.....	27
8.	ANEXE	28
8.1.	Anexa 1 – Module de securitate.....	28
8.2.	Anexa 2 – Motorul de navigare SPA și afișare	34
8.3.	Anexa 3 – Module de design CSS3.....	37
8.4.	Anexa 4 – Tabele din baza de date.....	40



1. INTRODUCERE ȘI PREGĂTIREA PROIECTULUI

1.1. Obiectivele proiectului

Obiectivul principal al proiectului constă în proiectarea și implementarea unei aplicații informatice destinate administrării și promovării unei unități de tip cafenea. Proiectul vizează eficientizarea interacțiunii dintre clienți și personalul administrativ prin intermediul unei platforme digitale integrate.

Obiectivele specifice ale lucrării sunt:

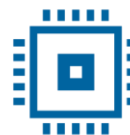
- **Accesibilitatea informației:** Furnizarea unui meniu digital interactiv care să permită clienților vizualizarea rapidă a produselor și a detaliilor aferente acestora.
- **Personalizarea experienței utilizatorului:** Implementarea unor funcționalități care să permită adaptarea serviciilor la preferințele individuale ale clienților.
- **Eficientizarea gestiunii interne:** Realizarea unui modul administrativ securizat, care să permită personalului gestionarea stocurilor, a produselor și a comenzilor într-un mod simplificat și performant.
- **Promovarea brandului:** Crearea unei interfețe grafice moderne și intuitive, menite să consolideze prezența online a cafenelei.

1.2. Justificarea temei

Acest proiect reprezintă o oportunitate strategică de dezvoltare personală și profesională, oferind cadrul necesar pentru transpunerea conceptelor teoretice în soluții software complexe, adaptate cerințelor pieței actuale. Dezvoltarea unei aplicații web pentru gestiunea unei cafenele reprezintă un studiu de caz ideal pentru aprofundarea conceptelor fundamentale de inginerie software.

Din perspectiva dezvoltării personale și profesionale, acest proiect oferă oportunitatea explorării următoarelor direcții tehnice:

- **Implementarea sistemelor de securitate:** Studiarea procesului de aplicare a protocoalelor de securitate cibernetică pentru protecția datelor sensibile.
- **Proiectarea bazelor de date cu specific de business:** Construcția unei baze de date centrate pe fluxurile de afaceri, punând accent pe integritatea și disponibilitatea informațiilor.
- **Analiza consumatorului și UX:** Proiectarea unei interfețe grafice intuitive, bazată pe analiza nevoilor utilizatorului final, pentru a oferi o experiență de navigare fluidă.
- **Arhitecturi moderne (SPA):** Utilizarea unei arhitecturi de tip „Single Page Application” pentru a asigura o performanță ridicată și o interactivitate sporită a platformei.



1.3. Tehnologii și instrumente utilizate

Pentru realizarea obiectivelor propuse, s-a optat pentru o listă tehnologică robustă, menită să asigure performanța și scalabilitatea aplicației web. Instrumentele selectate permit o dezvoltare eficientă, de la gestionarea bazei de date până la optimizarea resurselor media.

1.3.1. Mediul de dezvoltare și servere:

- **Visual Studio Code:** Utilizat ca mediu de dezvoltare integrat (IDE) principal, datorită extensibilității și suportului avansat pentru limbajele de programare folosite.
- **AMPPS:** Soluția software de tip *stack* utilizată pentru furnizarea serviciilor de infrastructură, incluzând serverul web **Apache** și serverul de baze de date **MySQL**.
- **PhpMyAdmin:** Instrumentul principal pentru administrarea și interogarea bazei de date prin interfață grafică.

1.3.2. Limbaje și tehnologii de programare:

- **PHP & SQL:** Utilizate pentru logica de tip *back-end* și pentru manipularea datelor prin sintaxe specifice bazelor de date relaționale.
- **JavaScript, HTML5 & CSS3:** Tehnologii fundamentale folosite pentru construcția interfeței utilizator și implementarea arhitecturii de tip SPA.
- **Batch Scripting & Python:** Folosite pentru automatizarea sarcinilor administrative, respectiv pentru procesarea și compresarea optimizată a imaginilor utilizate în aplicație.

1.3.3. Instrumente și biblioteci auxiliare:

- **FontAwesome:** Pentru integrarea unei iconografii unitare și moderne.
- **QR Generator:** Implementat pentru facilitarea accesului rapid al clienților la meniul digital.
- **Base64 Encoding/Decoding:** Tehnici utilizate pentru manipularea securizată sau optimizată a datelor în cadrul fluxurilor de informații.
- **SendMail (stdin):** Modul utilizat pentru gestionarea notificărilor și a comunicării automate prin e-mail.

2. STADIUL ACTUAL ȘI ANALIZA TEORETICĂ

2.1. Descrierea stadiului actual în aria temei proiectului

Zi de zi avansurile tehnologice digitalizează lumea din jurul nostru, împingând dezvoltarea tuturor sectoarelor. Sectorul HoReCa reprezintă, probabil, unul dintre segmentele cele mai puternic influențate de această tranziție digitală, fiind nevoit să identifice constant metode inovatoare pentru a atrage și a fideliza consumatorii. În acest context competitiv, succesul unei afaceri depinde în mare măsură de capacitatea de a oferi servicii într-un mod remarcabil, intuitiv și, mai ales, ușor accesibil prin intermediul platformelor online.

Implementarea unei aplicații web bazate pe o arhitectură de tip „Single Page Application” (SPA) aduce utilizatorului un acces direct și fluid la serviciile oferite de afacere. Această abordare arhitecturală asigură o interfață grafică plăcută și o experiență de navigare superioară oferind interactivitate superioară.



Totodată, pe măsură ce publicul larg devine tot mai familiarizat cu procesul de digitalizare, se observă o creștere a nevoilor privind securitatea cibernetică. Este imperativ ca orice soluție modernă să pună un accent deosebit pe mecanismele de protecție a datelor pentru a evita situațiile neprevăzute și pentru a garanta siguranța informațiilor procesate în mediul virtual, accentuat în cadrul proiectului nostru.

2.2. Studiul algoritmilor clasici utilizați în domeniu

Accentul aplicațiilor din domeniu este în general pe viteză, securitate și experiență fluidă pentru utilizator, astfel se renunță la algoritmi exotici în favoarea algoritmilor clasici bine aleși, o optimizare practică, ducând la o complexitate mică cu o eficiență mare.

În contextul securității datelor, standardul actual este reprezentat de algoritmul **SHA-256**, utilizat pentru hashing-ul parolelor. Acesta asigură integritatea informațiilor printr-un timp de procesare direct proporțional cu lungimea datelor de intrare, având o complexitate temporală definită de relația:

$$(I) T(n) = O(n)$$

unde n reprezintă lungimea parolei .

Pentru gestionarea elementelor din interfață, precum listele de produse, se utilizează algoritmi de căutare și sortare. **Căutarea liniară** este aplicată în cazul seturilor de date neordonate, având o complexitate temporală de:

$$(II) T(n) = O(n)$$

și o complexitate spațială de:

$$(III) S(n) = O(1)$$

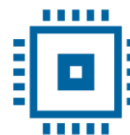
În situația în care listele sunt deja sortate, se preferă **căutarea binară**, care oferă o eficiență superioară prin complexitatea:

$$(IV) T(n) = O(\log n)$$

Organizarea produselor în funcție de diverse criterii (preț, categorie) se realizează prin algoritmi de tip **QuickSort** sau **MergeSort**, care garantează un timp de execuție optim pentru aplicații web:

$$(V) T(n) = O(n \log n)$$

Nu în ultimul rând, pentru a asigura navigarea între pagini fără reîncărcare (reload), specifică arhitecturii SPA, se utilizează concepte de **Graph Traversal**. Aceasta presupune modelarea aplicației ca un graf de stări, unde tranzițiile sunt gestionate printr-o formă simplificată de **DFS (Depth-First Search)** pentru a menține istoricul și starea componentelor.



2.3. Analiza comparativă a soluțiilor existente

Societatea post 2021 a observat o creștere exponențială a serviciilor software de livrare și rezervări precum și a tendinței de a digitaliza meniul tradițional. O analiză a soluțiilor existente, precum marile platforme de agregare (de exemplu, Glovo sau UberEats) sau aplicațiile dedicate ale marilor lanțuri de cafenele, evidențiază un contrast între robustețea sistemelor comerciale și flexibilitatea prototipurilor personalizate.

Soluțiile comerciale de amploare oferă avantaje clare în ceea ce privește stabilitatea, suportul tehnic și prezența pe piața HoReCa ce au devenit noile standarde, însă prezintă adesea limitări legate de costuri ridicate de implementare, comisioane și rigiditatea interfețelor ce nu permit o personalizare a identității micilor afaceri. În contrast, proiectul nostru propune o arhitectura de tip „Single Page Application” (SPA) care prioritizează viteza de răspuns și o experiență de utilizare fluidă, personalizată nevoilor fiecărei afaceri mici, fără a impune barierele financiare ale unei licențe proprietare.

Spre deosebire de sistemele monolitice care necesită reîncărcarea paginilor la fiecare interacțiune, prototipul nostru utilizează tehnologii precum JavaScript și PHP pentru a asigura o comunicare asincronă cu serverul, aspect ce îmbunătățește considerabil rata de retenție a utilizatorilor. Mai mult, în timp ce securitatea multor soluții accesibile este adesea lăsată pe plan secund, proiectul actual pune un accent deosebit pe integrarea algoritmilor de hashing și pe protecția datelor.

3. PROIECTAREA SISTEMULUI/APLICAȚIEI

3.1. Arhitectura generală a sistemului

Proiectarea aplicației are la bază o arhitectură de tip SPA structurată pe trei piloni principali: afișajul (Front-end), logica aplicației (Back-end) și Baza de date.

Front-end-ul, construit prin intermediul HTML5, CSS3 și JavaScript, constă în interfața grafică și în captarea evenimentelor (precum interacțiunea cu butoanele sau formularele) generate de utilizator. Această componentă reprezintă nucleul arhitecturii SPA. Conform definiției [\[1\]](#), o aplicație SPA este o aplicație web ce interacționează cu utilizatorul prin rescrierea dinamică a paginii web cu informații noi preluate de pe serverul web, contrar metodei obișnuite de a încărca întregi pagini noi, cu scopul de a face pagina cât mai apropiată de o aplicație nativă.

Back-end-ul, logica aplicației, este susținut de limbajul PHP și rulat pe un server Apache prin intermediul platformei AMPPS. Rolul logicii aplicației este a procesa „request”-urile utilizatorilor, de a valida datele și de a le comunica bazei de date. Totodată, avem funcția de trimitere a mail-urilor (sistemul de newsletter și de recuperare a parolei) și funcțiile Python folosite pentru optimizarea paginii (compresia imaginilor și conversia la extensia webp).

Stocarea informațiilor este realizată într-o bază de date relațională MySQL, administrată prin PhpMyAdmin. Aceasta gestionează datele de identificare a utilizatorilor, produsele și configurările de sistem ce permit administrarea dinamică a cafenelei.

Transferul de informații dintre acești piloni se realizează prin fluxuri de date securizate, folosind sintaxe SQL pentru interogări și protocoale standard, asigurând integritatea datelor.

3.2. Proiectarea bazei de date / Structuri de date utilizate

Aplicația noastră utilizează o bază de date relațională complexă, structurată pentru a asigura integritatea datelor și un acces rapid la informație. Arhitectura bazei de date este centrată în jurul utilizatorilor, integrând în jurul acestuia elemente precum comenzi, rezervări și servicii de cont.

Principalele entități identificate în structura bazei de date sunt:

- **Gestionarea Utilizatorilor și Securitate:** Tabelul *users* stochează informațiile de identificare și rolurile (client, administrator, angajat). Securitatea este întărită prin tabelele *password_resets* și *user_tokens*, care gestionează sesiunile și recuperarea accesului. [\[Anexa 4, Tabelul users\]](#)
- **Catalogul de Produse:** Tabelul *products* conține nomenclatorul de produse, incluzând detalii precum ingrediente, preț, disponibilitate și categorii (cafea, ceai, răcoritoare etc.). [\[Anexa 4, Tabelul products\]](#)
- **Sistemul de Comenzi și Rezervări:** Fluxul operațional este asigurat prin tabelele *orders* și *order_items* pentru achiziții, respectiv *tables* și *reservations* pentru gestionarea locurilor în cafenea. [\[Anexa 4, Tabelul orders\]](#)
- **Configurări și Marketing:** Administrarea dinamică se realizează prin *global_settings*, *settings* și *slider_images*, în timp ce istoricul email-urilor de tip newsletter este păstrat în *newsletters_history*.

Pentru o viziune de ansamblu asupra relațiilor dintre tabele, structura este prezentată în schema de mai jos:

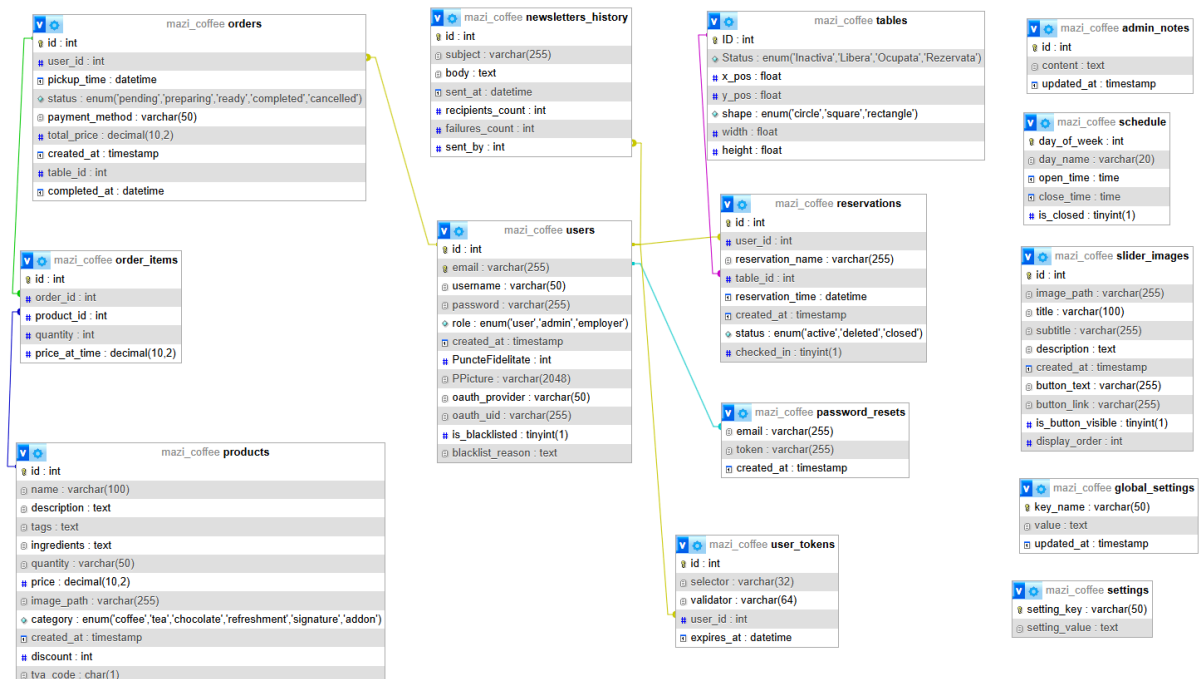


Fig. 3.1. Schema conceptuală a bazei de date (Diagrama ER)



3.3. Descrierea algoritmilor utilizați sau proiectați

3.3.1. Algoritm de Securitate și Criptare (Hibrid)

Constă în sistem complex de protejare a datelor sensibile (ex. plăți), inspirat din standardele bancare. Acest algoritm este de tip hibrid deoarece combină doi algoritmi. Criptarea asimetrică (RSA - cheie publică/privată) este foarte sigură, dar extrem de lentă pentru volume mari de date. Criptarea simetrică (AES - parolă unică) este rapidă, dar e riscant să trimiți "parola" prin internet. [\[Anexa 1 – Decriptare & Validare\]](#)

Acesta funcționează în patru pași:

1. **Generarea Cifrelor** (Client): Browserul creează spontan o cheie AES unică (un șir de 256 de biți) doar pentru această comandă. Viteza este instantanee.
2. **Criptarea Datelor**: Datele cardului sunt amestecate folosind această cheie AES rapidă. Acum avem un "seif" (datele criptate) și o "cheie" (cheia AES).
3. **Împachetarea Cheii** (RSA): Luăm "cheia" AES și o băgăm într-un plic digital sigilat cu Cheia Publică a serverului (RSA). Acest plic poate fi deschis doar cu Cheia Privată a serverului, care nu părăsește niciodată serverul.
4. **Transport**: Trimitem la server "seiful" + "plicul cu cheia". Nimeni de pe traseu nu poate deschide plicul, deci nimeni nu poate deschide seiful.

3.3.2. Algoritm de Detectare a Conflictelor (Suprapunerea Intervalelor)

Este metoda prin care se previne suprapunerea a doua rezervări făcute de clienți diferiți, verificând o intersecție de intervale. [\[Anexa 1 – Suprapunerea Intervalelor\]](#)

Avem **Intervalul A** (intervalul existent):

$$(VI) [Ora_Start, Ora_Sfârșit + 80\ minute]$$

,unde 80 de minute reprezintă 1 oră durată și 20 min pauză de debarasare.

Avem și **Intervalul B** (intervalul nou):

$$(VII) [Ora_Nouă_Start, Ora_Nouă_Sfârșit) + 60\ minute]$$

O suprapunere există dacă:

$$(VIII) Ora_Start_A < Ora_Sfârșit_B$$

și

$$(IX) Ora_Start_B < Ora_Sfârșit_A$$

În cod, am simplificat acest calcul prin diferențe: Există conflict dacă diferența dintre ore este între -80 minute și +60 minute.



3.3.3. Algoritmul „Debounce” (Lazy search/ Căutare leneșă)

Conform literaturii de specialitate [2], algoritmul debounce este o metodă de optimizare software utilizată pentru a asigura faptul că o funcție costisitoare sau o cerere către server nu este executată de mai multe ori într-un interval scurt de timp, în urma unor evenimente declanșate succesiv. În esență, acesta grupează o serie de apeluri secvențiale într-un singur eveniment, prin impunerea unei perioade de latență (pauză) necesare pentru validarea intenției utilizatorului. [\[Anexa 1- Algoritmul „Debounce”\]](#)

În cadrul proiectului, acest algoritm este folosit în motorul de căutare din meniu pentru a gestiona interacțiunea cu tastatura, funcționând după următorul mecanism:

1. **Inițializarea:** La prima apăsare a unei taste, sistemul pornește un cronometru setat la pragul de 300ms.
2. **Resetarea:** Orice apăsare ulterioară survenită în interiorul ferestrei de 300ms resetează cronometrul, amânând execuția.

$$(X)T_{total} = T_{pauză} + \Delta t$$

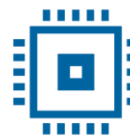
3. **Validarea:** Cererea unică către server (prin PHP) este expediată doar atunci când pauza dintre apăsări depășește pragul stabilit, confirmând finalizarea acțiunii de tastare.”

3.3.4. Algoritmul „Hydration” & SPA Routing

Conceptul de **Hydration** (Hidratare) și mecanismele de **SPA Routing** [3] reprezintă fundamentul arhitectural prin care o aplicație web simulează comportamentul unei aplicații native, eliminând tranzițiile vizuale disruptive asociate încărcării tradiționale a paginilor. [\[Anexa 2 - Algoritmul „Hydration” & SPA Routing\]](#)

În cadrul proiectului nostru, acest proces a fost implementat printr-o tehnică de **Hot-Swapping**, care urmează un flux algoritmic structurat pentru a asigura fluiditatea interfeței:

1. **Interceptarea Evenimentelor (Event Interception):** Motorul de JavaScript interceptează acțiunea de „click” asupra elementelor de navigare, blocând comportamentul implicit al browserului de a reîmprospăta întreaga pagină.
2. **Preluarea Asincronă (Background Fetch):** În timp ce interfața curentă rămâne activă și vizibilă, aplicația solicită serverului (PHP) strict fragmentul de conținut aferent noii pagini, optimizând astfel consumul de date.
3. **Pre-Procesare și Controlul Stilului (Anti-FOUC):** Pentru a evita fenomenul de *Flash of Unstyled Content* (FOUC), algoritmul verifică și descarcă în memorie fișierele CSS noi înainte de afișare.



-
4. **Actualizarea Dinamică (UI Surgery):** După descărcarea completă a resurselor, containerul principal de tip <div> este înlocuit instantaneu cu noul conținut. Această tranziție elimină „ecranul alb” și asigură o experiență de utilizare continuă.

3.3.5. Algoritm fiscal (TVA multiplu)

Pentru gestionarea coșului de cumpărături și asigurarea conformității cu legislația fiscală privind produsele alimentare, s-a implementat un algoritm de **Agregare Fiscală bazat pe tehnica Bucket Sort**. Această abordare este necesară deoarece aplicația trebuie să proceseze simultan cote de TVA diferite (de exemplu, 21% pentru băuturi pe bază de cafea și 11% pentru produse de patiserie sau sandvișuri), evitând aplicarea unei cote unice eronate asupra întregului total. [\[Anexa 1 – Algoritm fiscal\]](#)

Funcționarea algoritmului în cadrul aplicației urmează următorii pași logici:

1. **Iterarea elementelor:** Algoritmul parcurge fiecare produs adăugat în coșul de cumpărături de către utilizator.
2. **Identificarea categoriei fiscale:** Pentru fiecare produs în parte, se citește atributul *tva_code* (A, B sau C) preluat din baza de date.
3. **Calculul taxei individuale:** Se extrage valoarea TVA-ului inclusă în prețul fiecărui produs utilizând formula matematică:

$$(XI) \text{ Valoare}_{TVA} = \text{Preț}_{brut} * \frac{Cota}{100+Cota}$$

4. **Agregarea (Bucket Sort):** Valorile rezultate nu sunt cumulate într-o variabilă globală, ci sunt adunate separat în variabile specifice fiecărui cod fiscal (ex: *Suma_TVA_A*, *Suma_TVA_B*).
5. **Finalizarea raportului:** La închiderea comenzii, aplicația nu afișează doar un total general, ci prezintă defalcăt sumă taxelor pentru fiecare cotă în parte.

3.3.6. Ripple Effect (efectul de undă)

Algoritm pentru efectul de "Ripple" (undă) este un exemplu clasic de geometrie aplicată în interfață pentru a crea iluzia fizică de interacțiune. [\[Anexa 1 – Ripple Effect\]](#)

În esență, algoritmul transformă coordonatele cursorului la momentul de click (relative la ecran) în coordonate relative la buton, pentru a poziționa și dimensiona un cerc care se expandează. La click, se dorește ca efectul (unda) să pornească exact de sub cursorul mouse-ului, nu din centrul butonului. Totuși, efectul trebuie să acopere tot butonul până la finalul animației:

1. **Calculul Diametrului:** Pentru ca unda să acopere tot butonul, indiferent unde se dă click (chiar și într-un colț), diametrul cercului trebuie să fie cel puțin egal cu cea mai mare latură a butonului:

$$(XII) \text{ Diametru} = \text{MAX}(\text{LățimeButon}, \text{ÎnălțimeButon})$$



-
2. **Calculul Coordonatelor Relative:** Browserul transmite poziția globală a mouse-ului (*ClientX*, *ClientY*) și poziția globală a butonului (*Rect.left*, *Rect.top*).

$$(XIII) \text{ PozitieX_Click} = \text{ClientX} - \text{Rect.left}$$

$$(XIV) \text{ PozitieY_Click} = \text{ClientY} - \text{Rect.top}$$

3. **Centrarea Cercului:** Elementele HTML se poziționează după colțul lor din stânga-sus. Dacă am pune cercul la *PozitieX_Click*, colțul cercului ar fi sub mouse, nu centrul lui. Pentru a alinia centrul cercului cu punctul de click, scădem raza.

$$(XV) \text{ Top} = \text{PozitieY_Click} - \text{Raza}$$

$$(XVI) \text{ Left} = \text{PozitieX_Click} - \text{Raza}$$

3.4. Diagrama claselor și a obiectelor (UML)

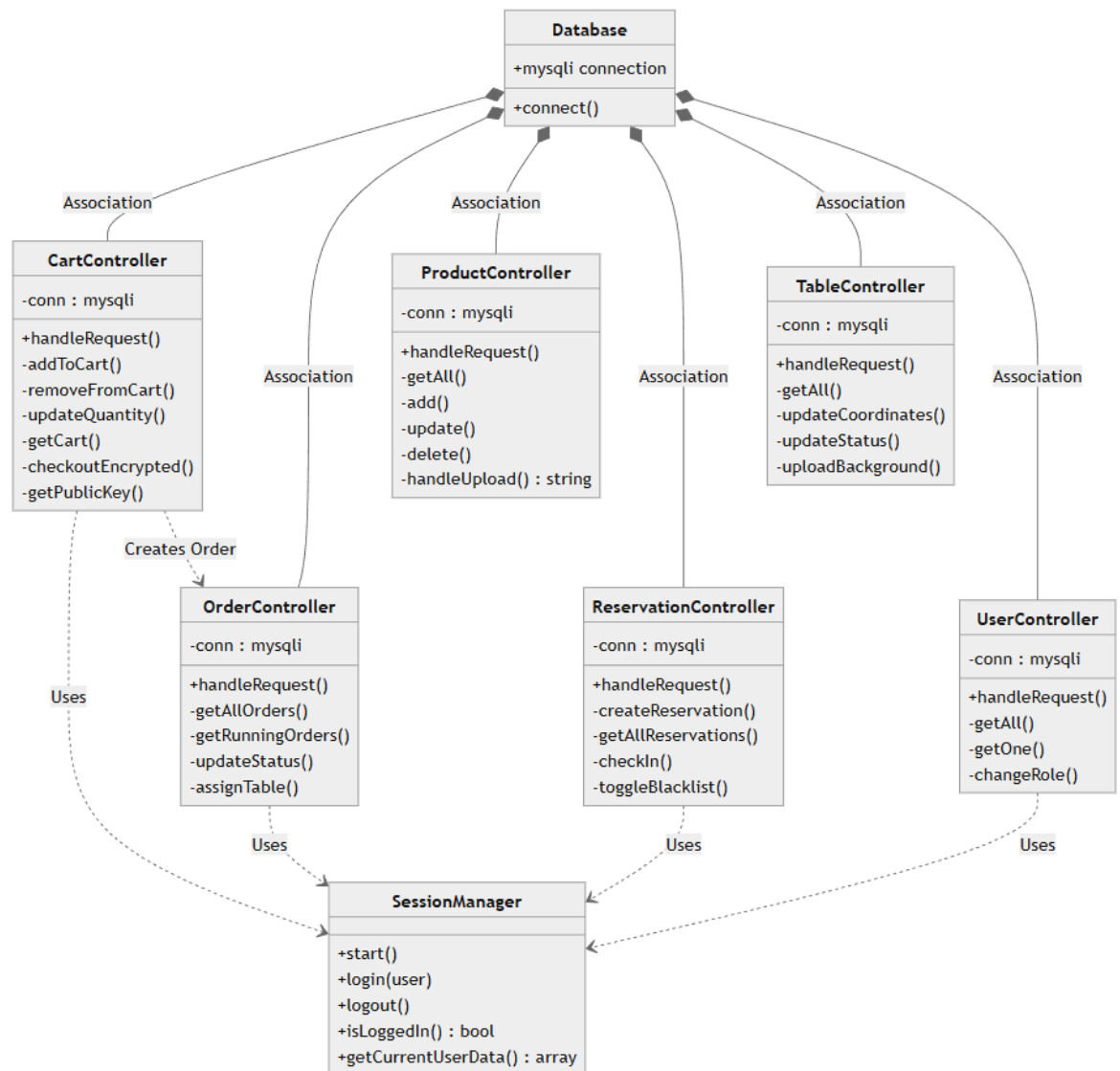


Fig. 3.1 Diagrama UML a componentei de control

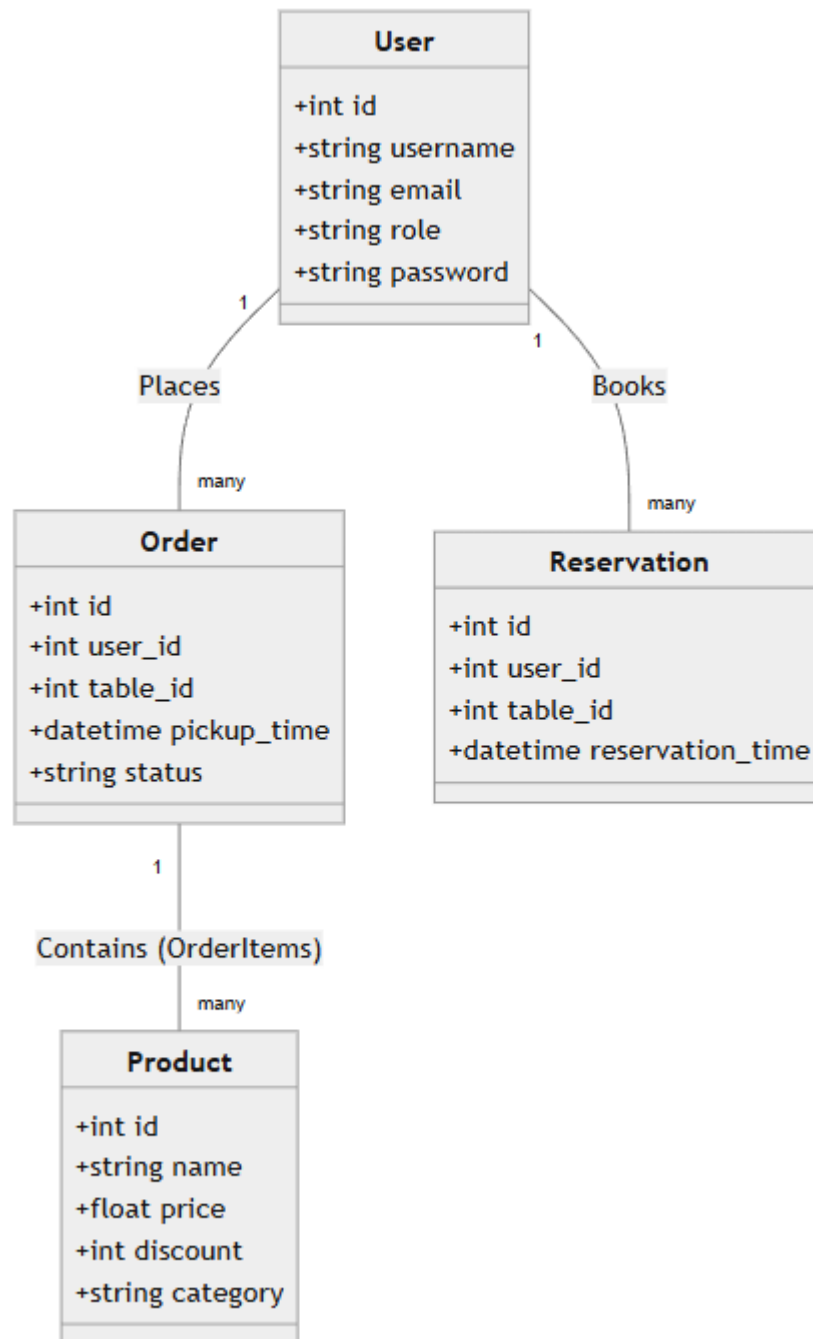
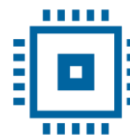


Fig. 3.2 Diagrama UML a componentei de date

Structura funcțională a backend-ului, așa cum este prezentată în **Fig. 3.1.** și **Fig. 3.2.**, este organizată pe trei paliere de responsabilitate:

- **Componenta de Control (Controller):** Aceasta reprezintă palierul central al logicii de business. Clasele de tip Controller (precum *ProductController*, *CartController* sau *ReservationController*) sunt responsabile pentru interceptarea cererilor de tip *AJAX*



venite din interfață, verificarea permisiunilor de acces prin intermediul clasei *SessionManager* și execuția operațiunilor solicitate.

- **Componenta de Vizualizare (View):** Reprezentată de interfața utilizator (Frontend), aceasta comunică direct cu controllerele pentru a actualiza dinamic conținutul paginii fără a reîncărca documentul HTML.
- **Componenta de Date (Model):** În arhitectura actuală, logica de date este încapsulată direct în controllere. Acestea interacționează cu clasa *Database* pentru a executa interogări SQL, urmând un model de tip *Active Record* simplificat, unde starea obiectelor (cum ar fi *User*, *Order* sau *Product*) este mapată direct pe structura tabelor din baza de date MySQL.

Relațiile de dependență sunt clar definite: controllerele depind de clasa *Database* pentru persistența datelor și utilizează *SessionManager* pentru a menține starea de autentificare și securitatea sesiunilor. De exemplu, *CartController* nu doar gestionează produsele din coș, ci colaborează cu *OrderController* pentru finalizarea procesului de achiziție și generarea entității de tip *Order*.

4. IMPLEMENTAREA SOFTWARE

4.1. Descrierea modulelor software importante

4.1.1. Module Backend (Controllere și Core)

Aceste module rulează la nivelul serverului PHP și gestionează logica de business, securitatea și interacțiunea cu baza de date MySQL:

- **Modulul de Tranzacționare (*CartController*):** Este responsabil pentru gestionarea coșului de cumpărături și calculul totalurilor financiare. Acesta implementează algoritmul de agregare a TVA-ului pe categorii fiscale (21% și 11%) și un proces critic de *checkout* securizat. Funcția *checkoutEncrypted()* asigură procesarea plăților printr-o decriptare hibridă (RSA + AES), în timp ce un sistem de validare verifică stocul în timp real înainte de finalizarea comenzii. [\[Anexa 1 – checkoutEncrypted\(\)\]](#)
- **Sistemul de Rezervări (*ReservationController*):** Administrează planificarea meselor utilizând un algoritm de tip *Sliding Window* pentru detectarea suprapunerilor temporale. Acesta garantează o pauză tehnică de 20 de minute între clienți și include un mecanism de *Auto-Cancel* pentru a gestiona automat situațiile de tip „No-Show”.
- **Managementul Comenzilor (*OrderController*):** Controlează întreg ciclul de viață al unei comenzi, de la starea de așteptare (*Pending*) până la finalizare (*Completed*). Modulul este integrat cu planul sălii, eliberând automat masa asociată atunci când nu mai există comenzi active pentru aceasta.
- **Planul Sălii (*TableController*):** Oferă administratorului o interfață vizuală pentru gestionarea locației. Acesta permite editarea coordonatelor carteziene ale meselor, redimensionarea acestora și personalizarea fundalului prin încărcarea schițelor arhitecturale.



-
- **Securitate și Sesiuni (*SessionManager*):** Reprezintă nucleul de protecție al aplicației. Implementează mecanisme de apărare împotriva atacurilor de tip *CSRF Protection* și un sistem de control al accesului bazat pe roluri (*RBAC*), restricționând accesul la modulele administrative în funcție de privilegiile utilizatorului (Admin, Employer, User).

4.1.2. Module Frontend (Client-Side)

Aceste componente rulează în browserul utilizatorului și sunt responsabile pentru experiența fluidă de utilizare (*User Experience*):

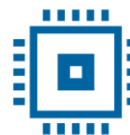
- **Motorul SPA (*router.js*):** Transformă navigarea într-o experiență continuă prin algoritmi de *Hydration* și *Preloading*. Acesta încarcă asincron conținutul paginilor noi, eliminând tranzițiile vizuale disruptive și gestionând istoricul browserului pentru butoanele *Back/Forward*. [\[Anexa 2 – Algoritmul „Hydration” & SPA Routing\]](#)
- **Interfața de Plăți (*payment.php*):** Utilizează *Web Crypto API* pentru a genera chei de criptare direct în memoria browserului. Datele sensibile sunt criptate la nivelul clientului înainte de a fi transmise prin rețea, asigurând un nivel înalt de confidențialitate.
- **Interfața Grafică (*ui.js* și *slider.js*):** Gestionează feedback-ul vizual și performanța afișării. Include algoritmi de tip *Ripple Effect* pentru interacțiunea cu butoanele și tehnici de *Lazy Loading* pentru optimizarea timpului de încărcare a imaginilor din meniuri și *slider*.

4.2. Funcțiile principale și parametrii utilizați

4.2.1. Funcții Backend (PHP Controllers)

Aceste funcții procesează cererile primite prin protocoalele HTTP (POST/GET) și interacționează direct cu baza de date MySQL:

- **`checkoutEncrypted($encryptedData, $iv, $encryptedKey)`:** Reprezintă cea mai complexă metodă din punct de vedere al securității, fiind responsabilă pentru orchestrarea decriptării hibride (RSA + AES). Aceasta primește datele sensibile criptate de client (`$encryptedData`), vectorul de inițializare (`$iv`) și cheia AES criptată cu cheia publică a serverului (`$encryptedKey`), returnând un obiect JSON cu statusul tranzacției.
 - **`createReservation($userId, $tableId, $reservationTime, $duration)`:** Gestionează logica rezervărilor, asigurând validarea strictă a disponibilității. Aceasta utilizează un buffer forțat de 20 de minute între rezervări pentru a asigura pregătirea mesei. Funcția apelează intern `checkAvailability()`, care execută o interogare SQL optimizată pentru detectarea conflictelor temporale.
 - **`updateCoordinates($id, $x, $y, $w, $h)`:** Permite administrarea vizuală a planului cafenelei. Parametrii `$x`, `y`, `w`, `h` sunt tratați ca valori procentuale (0-100%), asigurând astfel un design responsiv al hărții interactive, indiferent de rezoluția ecranului administratorului. [\[Anexa 2 – `updateCoordinates\(\)`\]](#)
-



-
- **getAll(\$category, \$searchQuery)**: Este funcția principală de furnizare a conținutului, permițând filtrarea produselor din meniu în funcție de categoriile fiscale definite sau de termenii de căutare introduși de utilizator. [\[Anexa 2 – getAll\(\)\]](#)

4.2.2. Funcții Frontend (JavaScript)

Aceste componente rulează direct în browserul utilizatorului, asigurând performanța specifică unei aplicații SPA:

- **loadPage(targetPage, pushState)**: Reprezintă motorul de navigare al aplicației. Acesta solicită conținutul paginii țintă și utilizează API-ul *history.pushState* pentru a actualiza adresa URL fără a declanșa reîncărcarea documentului, menținând starea aplicației.
- **encryptData(sensitiveJson, sessionId)**: Localizată în modulul de plăți, această funcție utilizează biblioteca nativă *window.crypto.subtle* pentru a securiza datele cardului. Aceasta transformă obiectul JSON în format *ArrayBuffer* cifrat, garantând că informațiile brute nu părăsesc niciodată mediul local al clientului.
- **showContentModal(htmlContent)**: O funcție utilitară de tip UI care gestionează ferestrele modale. Aceasta permite injectarea dinamică a fragmentelor de cod HTML primite de la server, facilitând interacțiuni precum încărcarea de fișiere sau confirmarea comenzilor. [\[Anexa 2 – showContentModal\(\)\]](#)

4.3. Detalii de implementare

Arhitectura proiectului Mazi Coffee este construită pe un model **Custom MVC (Model-View-Controller)** simplificat, conceput pentru a oferi performanță maximă fără dependența de framework-uri externe. Această abordare permite un control granular asupra resurselor și o securitate sporită.

4.3.1. Arhitectura și Design Patterns

Sistemul utilizează un mecanism de **Front Controller Hibrid**, unde navigarea este gestionată la nivel de client prin *router.js*, în timp ce fișierele PHP din directorul *controllers/* acționează strict ca puncte finale (endpoints) pentru API.

În dezvoltarea codului, au fost implementate următoarele șabloane de proiectare (Design Patterns):

- **Singleton (Static Wrapper)**[\[4\]](#): Clasa *SessionManager* garantează o stare globală unică a sesiunii prin metode statice, prevenind conflictele de autentificare.
- **Active Record Simplificat**[\[5\]](#): Proprietățile obiectelor de business sunt mapate direct pe coloanele bazei de date, facilitând operațiunile de tip CRUD.
- **Strategy Pattern**: Utilizat în *CartController* pentru a aplica metode diferite de taxare în funcție de codul TVA, asigurând o scalabilitate ridicată în cazul modificărilor legislative.



4.3.2. Securitate și Gestiunea Datelor

Securitatea este pilonul central al implementării, utilizând extensia nativă **OpenSSL** pentru criptare hibridă:

- **Criptare RSA:** Se folosește padding-ul **OAEP** pentru a preveni atacurile de tip *Padding Oracle* în timpul decriptării cheilor simetrice.
- **AES-256-GCM:** Datele sensibile sunt procesate în modul GCM (Galois/Counter Mode), care oferă atât confidențialitate, cât și autentificarea datelor prin utilizarea unui „tag” de integritate.
- **Protecție SQL:** Integritatea bazei de date este garantată prin utilizarea exclusivă a instrucțiunilor pregătite (prepared statements), eliminând riscul atacurilor de tip *SQL Injection*.

4.3.3. Ingineria Frontend și Starea Aplicației

La nivel de Frontend, aplicația utilizează **Sesiuni Hibride**: starea de autentificare este menținută pe server, în timp ce coșul de cumpărături este gestionat local via `localStorage` pentru a reduce încărcarea serverului și a asigura persistența datelor.

Codul JavaScript este structurat modular (ES6), utilizând `import/export` pentru a menține componentele (precum `router.js` sau `cart.js`) independente și ușor de testat. De asemenea, aspectul vizual este controlat prin **Variable CSS** definite în `:root`, permițând modificări globale de tematică prin ajustarea câtorva parametri centralizați.

4.4. Descrierea interfeței grafice (GUI)

Interfața grafică a aplicației Mazi Coffee a fost proiectată pentru a oferi o experiență de utilizare fluidă, intuitivă și estetică, adaptată identității vizuale a unui spațiu modern de tip cafenea. Designul urmează principiile de *Responsive Web Design*, asigurând o funcționare optimă pe o gamă largă de dispozitive, de la terminale mobile la sisteme desktop.

4.4.1. Identitate vizuală și ergonomie

Paleta cromatică este centrată pe tonuri calde, specifice domeniului HoReCa. S-a utilizat un maro închis pentru elementele de contrast și butoanele principale, completat de un bej-crem pentru fundaluri, generând o atmosferă primitoare. Tipografia modernă, de tip *sans-serif*, asigură o lizibilitate ridicată, în timp ce feedback-ul vizual pentru succes sau erori este integrat organic în design, evitând ferestrele de alertă standard care pot întrerupe fluxul de navigare.

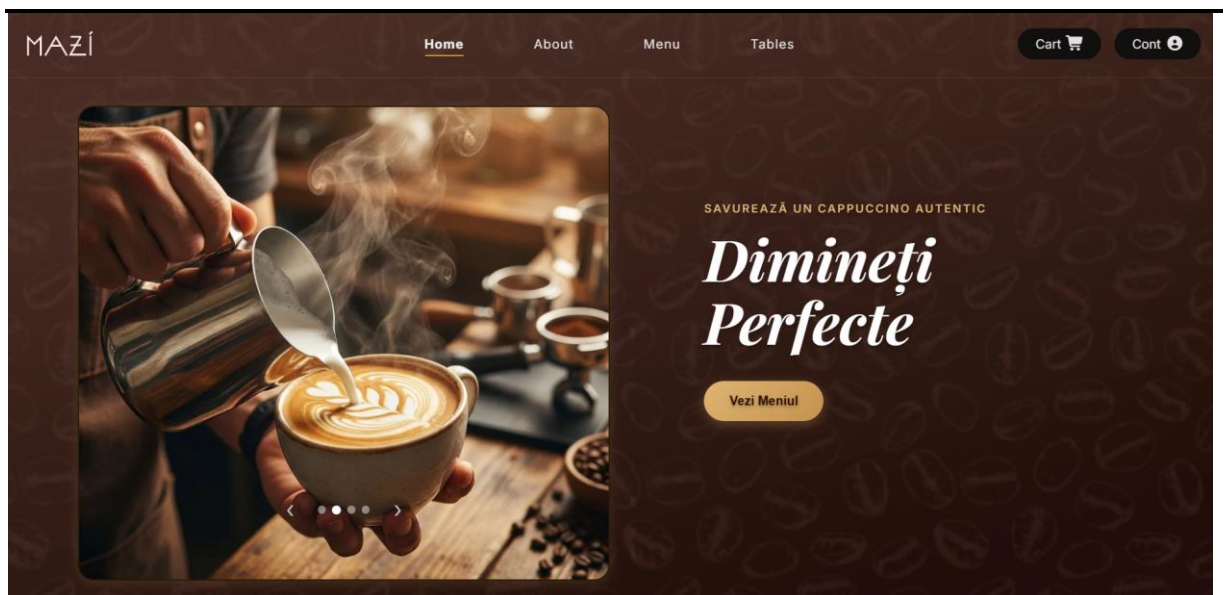


Fig. 4.1 Home Page

4.4.2. Organizarea layout-urilor principale

Aplicația este structurată pe trei paliere vizuale majore, fiecare fiind optimizat pentru sarcini specifice:

- **Interfața Publică (Client):** Pagina de pornire (Fig. 4.1.) utilizează un *Hero Slider full-width* pentru promovarea ofertelor, urmat de un meniu digital organizat sub formă de grilă responsivă. Produsele sunt prezentate prin carduri interactive care afișează prețurile, etichetele de reducere și oferă acces rapid la coșul de cumpărături prin intermediul unui *Sidebar* de filtrare. [\[Anexa 3 – Slider\]](#)
- **Modulul de Plată (Immersive Mode):** Pentru a minimiza distragerile în etapa critică a achiziției, interfața adoptă un stil minimalist, eliminând elementele de navigare secundare. Elementul central este un card bancar virtual 3D, care se actualizează în timp real pe măsură ce datele sunt introduse, oferind o confirmare vizuală a validării prin schimbări cromatice instantanee.(Fig 4.2.) [\[Anexa 3 – Card\]](#)
- **Panoul de Administrare:** Se remarcă prin harta interactivă a sălii (*Floor Plan*), un editor vizual de tip *Drag & Drop*. Acesta permite gestionarea meselor și monitorizarea stării acestora (Liberă, Ocupată, Rezervată) prin intermediul unui cod de culori intuitiv. (Fig. 4.3.) [\[Anexa 3 – Floor Plan\]](#)

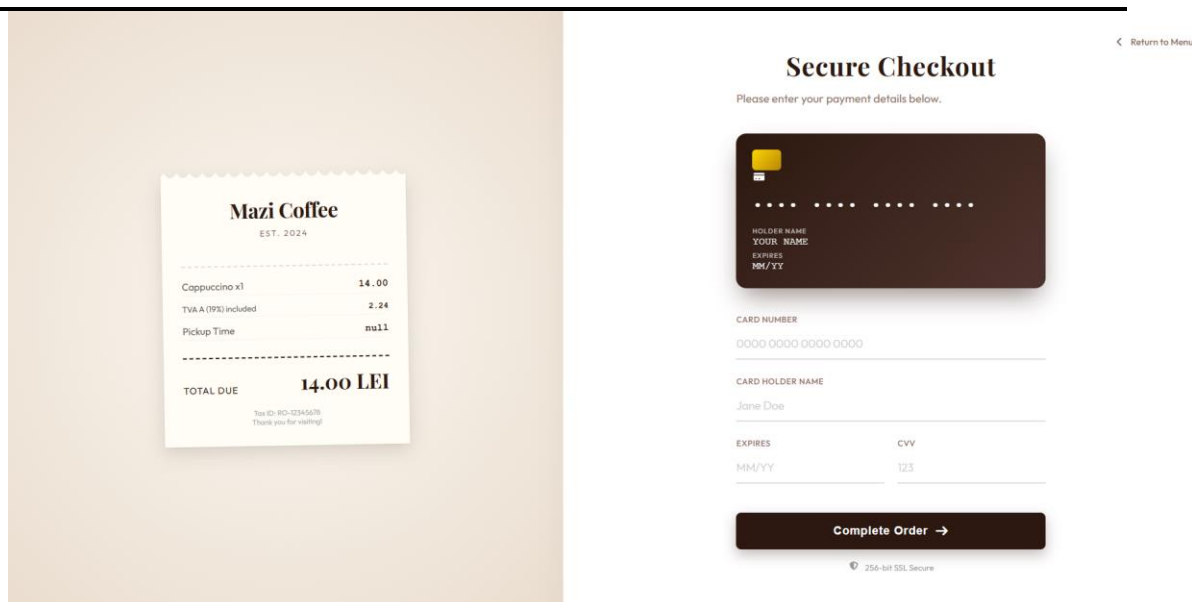


Fig. 4.2 Terminal de plată

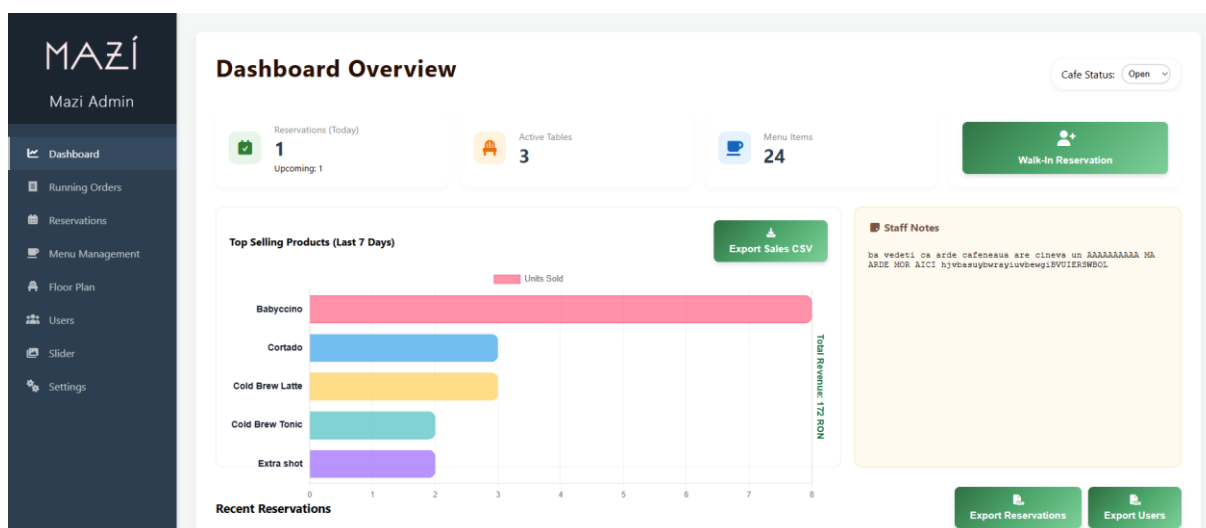


Fig. 4.3. Admin page, Dashboard

4.4.3. Elemente interactive și optimizări vizuale

Pentru a crește nivelul de profesionalism al interfeței, au fost implementate o serie de tehnici avansate de interactivitate:

- **Ripple Effect:** Animații bazate pe calcul geometric care oferă feedback tactil vizual la interacțiunea cu butoanele.
- **Skeleton Loading:** O tehnică de optimizare a percepției vitezei, prin care utilizatorul vede structura conținutului sub formă de forme pulsânde înainte de încărcarea completă a datelor, eliminând senzația de „ecran alb”. (Fig. 4.4.)

- **Toast Notifications:** Notificări non-intruzive care confirmă acțiunile utilizatorului (ex: „Produs adăugat în coș”), menținând continuitatea navigării.

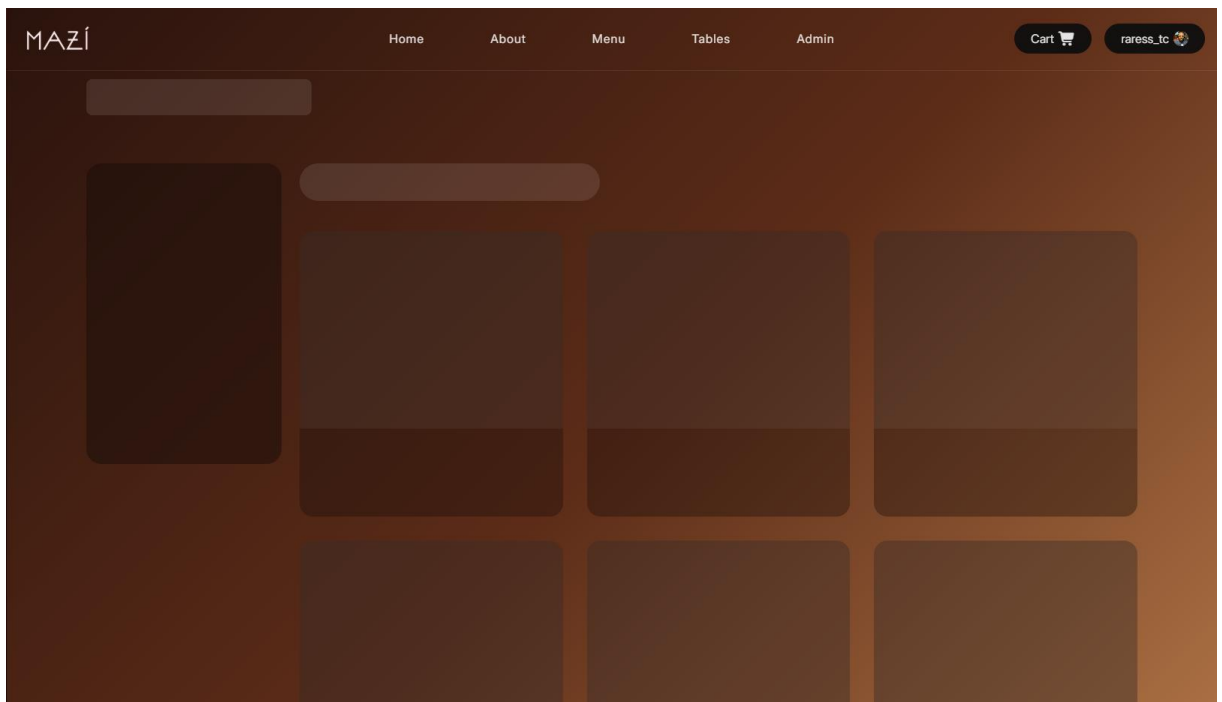


Fig. 4.4 Skeleton-ul paginii de menu

5. TESTARE ȘI REZULTATE

5.1. Scenarii de testare

Validarea funcționalității sistemului Mazi Coffee a fost realizată printr-o serie de scenarii de testare riguroase, menite să verifice atât fluxurile operaționale standard, cât și mecanismele de securitate și logică de business.

A. Scenariul: Comandă la Masă (Validare prin Token QR)

Acest scenariu vizează experiența unui client care utilizează sistemul de comandă rapidă prin scanarea unui cod QR, fără necesitatea autentificării prealabile.

- **Mecanism de declanșare:** Utilizatorul accesează URL-ul generat de codul QR, care conține un parametru de tip `table_token` codificat Base64. (Fig. 5.1.)
- **Validarea la nivel de Backend:** Clasa *CartController* interceptează cererea, decriptează token-ul și extrage `table_id`. Se verifică integritatea timbrului de timp pentru a preveni atacurile de tip *replay* (folosirea unor coduri expirate).



- **Procesul de Guest Checkout:** Sistemul detectează prezența token-ului valid și activează automat un cont temporar de tip „Oaspete”. Comanda este procesată și asociată mesei respective în baza de date. (Fig 5.2.)
- **Rezultat:** Comanda este înregistrată cu succes, iar în panoul administratorului starea mesei se schimbă automat în „Ocupată” (marcată cromatic cu roșu pe harta interactivă).



Fig.5.1. QR code pentru Table 1

Order #43

raess_tc

PENDING

CASH

- 3x Cortado
- 8x Babyccino
- 2x Cold Brew Tonic
- 3x Cold Brew Latte
- 2x Extra shot

COLLECT: 172.00 RON

Created: 07:19 AM

Table Order

Total: 172.00 RON

Pending

Prep

Ready

Serving for:

Table 1 (Ocupata)

✓ Complete & Clear

Fig.5.2. Comandă făcută direct la masă cu plată cash

B. Scenariul: Plată Securizată (Criptare Hibridă)

Testul a urmărit integritatea și confidențialitatea datelor bancare în timpul tranzacției.

- **Acțiune:** Utilizatorul introduce datele cardului și inițiază plata.
- **Procesare:** La nivel de client (browser), se generează o cheie simetrică AES unică. Datele sunt cifrate și transmise către server împreună cu cheia AES, aceasta din urmă fiind protejată prin cheia publică RSA a serverului.
- **Validare:** Serverul utilizează cheia privată RSA pentru a recupera cheia simetrică, decriptând ulterior datele plății.
- **Rezultat:** Tranzacția este finalizată cu succes fără ca informațiile în clar (număr card, CVV) să fi fost expuse în fluxul de rețea, demonstrând reziliența sistemului la interceptări. (Fig. 5.3.)

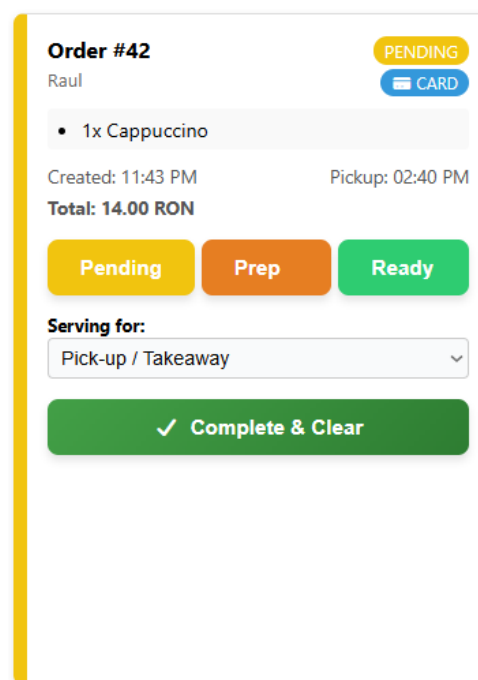


Fig. 5.3. Comandă făcută prin aplicație pentru pick-up la ora 14:40

C. Scenariul: Rezervare cu Suprapunere (Logică de Business)

Acest scenariu testează algoritmul de detectare a conflictelor temporale în gestionarea rezervărilor.

- **Context:** Există o rezervare activă pentru Masa 1 în intervalul 14:00 - 15:00. Un alt client încearcă să rezerve aceeași masă la ora 14:40.
- **Execuție:** Algoritmul din *ReservationController* calculează ferestrele de disponibilitate aplicând un buffer de siguranță de 20 de minute pentru pregătirea mesei. Astfel, intervalul ocupat este considerat [14:00, 15:20].
- **Rezultat:** Sistemul identifică suprapunerea cu noul interval solicitat [14:40, 15:40] și respinge automat cererea, oferind utilizatorului sugestii alternative (o altă masă sau primul interval disponibil după ora 15:20).



5.2. Analiza performanțelor aplicației

Evoluția sistemului a fost monitorizată constant, comparând performanțele versiunii inițiale cu cele ale versiunii finale, optimizate. Analiza s-a concentrat pe doi piloni fundamentali ai experienței utilizatorului (*User Experience*): reducerea timpului de încărcare a conținutului principal (**LCP** - *Largest Contentful Paint*)[\[6\]](#) și minimizarea blocajelor la interacțiune (**INP** - *Interaction to Next Paint*)[\[7\]](#).

A. Optimizarea paginii principale (Homepage)

În versiunea preliminară, pagina principală prezenta latențe ridicate la încărcare deoarece browserul încerca să descarce simultan toate resursele (imagini de înaltă rezoluție pentru slider, scripturi auxiliare și fonturi externe). Acest comportament bloca procesul de randare până la finalizarea transferului de date.

Pentru remedierea acestor deficiențe, au fost aplicate următoarele strategii:

- **Lazy Loading pentru conținutul media:** Scriptul `slider.js` a fost restructurat pentru a încărca inițial doar prima imagine din carusel. Imaginile secundare sunt solicitate asincron, doar după ce elementele critice ale paginii au fost deja randate.
- **Prioritizarea execuției scripturilor:** S-a utilizat atributul `defer` pentru toate modulele JavaScript neesențiale, permițând afișarea instantanee a structurii HTML și îmbunătățirea scorului de performanță în instrumente de audit precum *Google Lighthouse*.

B. Eficientizarea navigării în modulul de meniu

Versiunea inițială a meniului digital utiliza metoda tradițională de reîncărcare completă a paginii la fiecare filtrare a produselor. Acest flux genera un consum inutil de lățime de bandă și un efect vizual perturbator de „pâlpăire” a ecranului (*Flash of White*).

Optimizările implementate au vizat fluidizarea interacțiunii:

- **Implementarea rutării de tip SPA:** Navigarea între categorii este gestionată acum de `router.js`, care interoghează asincron serverul și injectează strict datele necesare în DOM, fără a reîmprospăta întreaga interfață.
- **Optimizarea formatelor de imagine:** Produsele sunt servite utilizând formatul modern `.webp`, ceea ce a dus la o reducere a consumului de date cu aproximativ **70%** pentru o sesiune de utilizare medie.

Concluzie privind performanța: Prin adoptarea arhitecturii asincrone și a metodelor de comprimare a resurselor, aplicația asigură o experiență de utilizare stabilă și rapidă, fiind optimizată inclusiv pentru dispozitive mobile cu conexiuni la internet limitate (3G/4G).

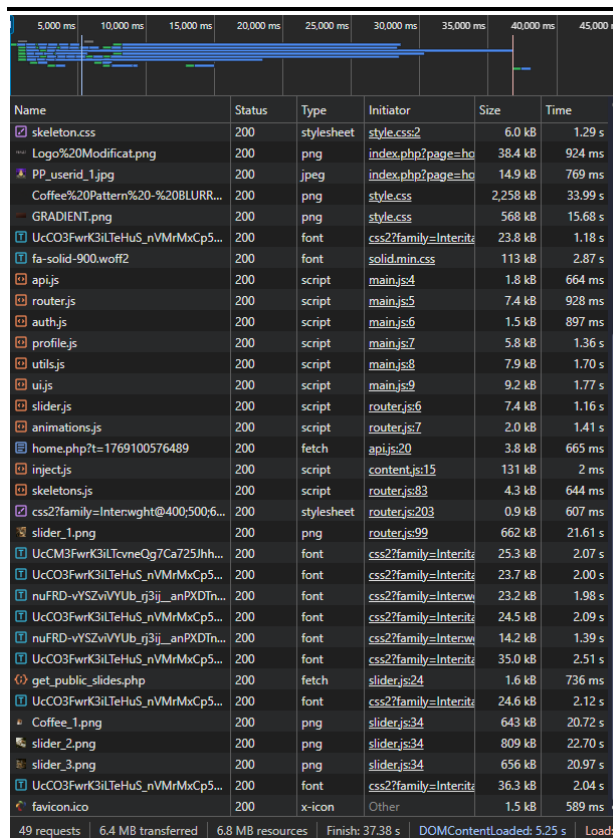


Fig. 5.4. Performanță veche home page

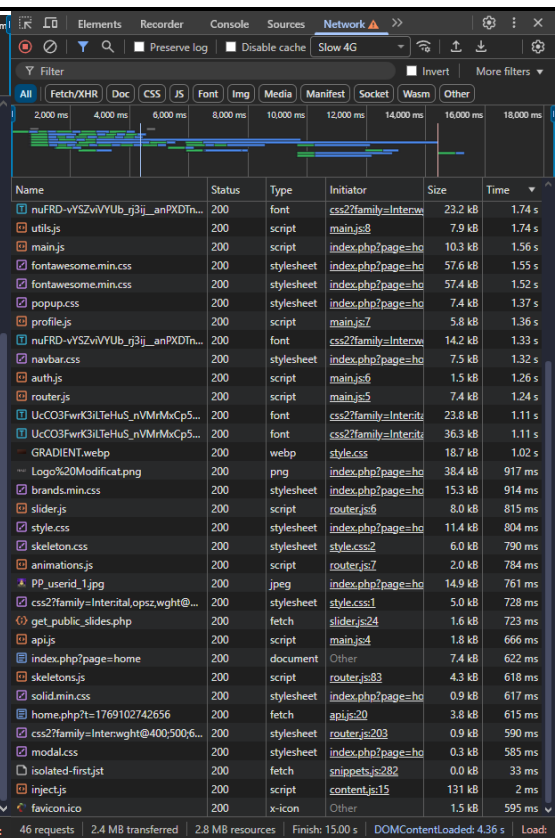


Fig. 5.5. Performanță nouă home page

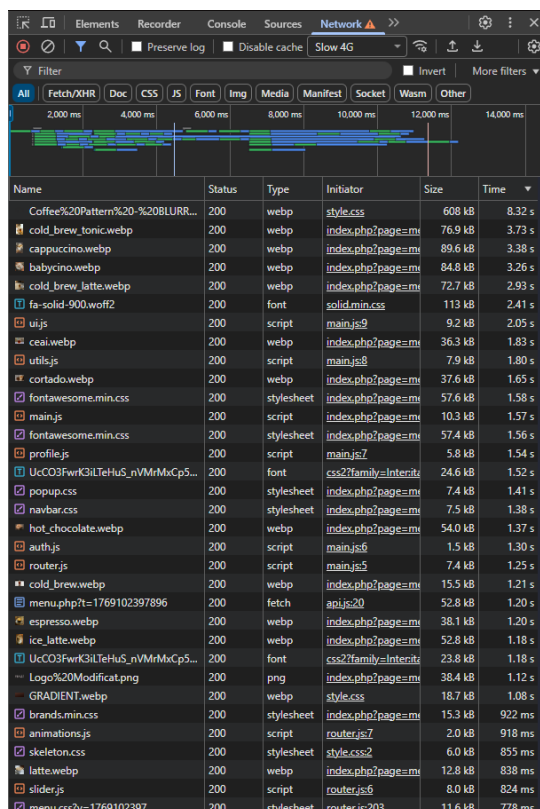


Fig. 5.6. Performanță veche meniu

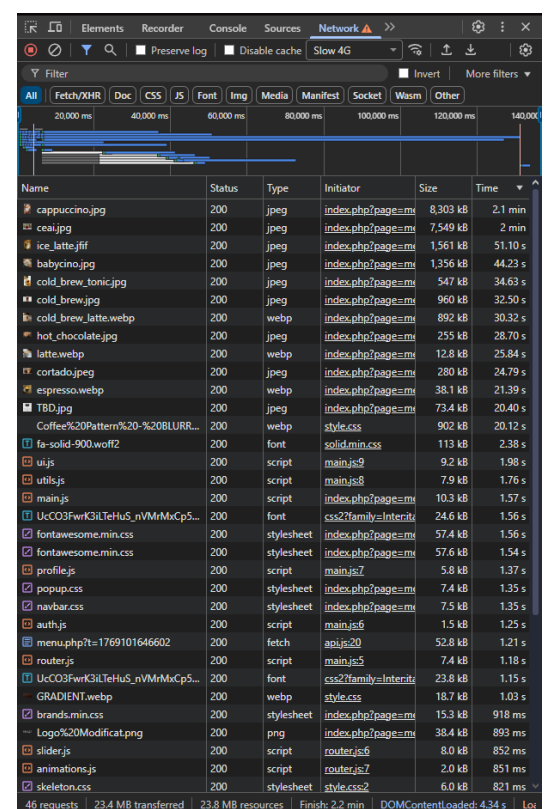


Fig. 5.7. Performanță nouă meniu



6. CONCLUZII

6.1. Concluzii generale

Proiectul de dezvoltare a unei aplicații web pentru cafeneaua Mazi Coffee a reprezentat oportunitatea perfectă de a studia, dezvolta și încerca algoritmi noi, într-un mediu real. De la securitate cibernetică până la manipularea datelor în interiorul bazei de date, proiectul a fost plin de încercări și reușite în materie de dezvoltare personală pentru întreaga echipă. Aplicarea algoritmilor ne-a schimbat perspectiva asupra securității și cât de multe vulnerabilități există într-o aplicație domestică. Ne-a mai învățat că este important să luăm în considerare performanța, soluția optimă și scalabilitatea/modularizarea proiectului la ori ce pas.

6.2. Elemente de originalitate

Printre elementele de originalitate se numără:

- Sistemul de comandă online pentru cafea take-away
- Afișarea layout-ului cafenelei cu prezentarea stării meselor
- Script de sincronizarea a bazei de date în interiorul echipei
- Exportarea datelor de interes

6.3. Direcții viitoare de dezvoltare

- Sistem de gestiune cu conexiune cu o casa de marcat
- Posibilitatea de plata cu POS
- Trecerea de la place holder la plata reală în interiorul site-ului
- Lansarea pe server/e hostat/e

6.4. Impresii

Considerăm că acest proiect a fost o idee inedită prin care am explorat mai mult ce am putea face în viitor. Oricare ar fi limbajul sau stilul de programare, certitudinea este că munca este o resursă esențială în a înțelege „Cum să programez?”. Fără să căutăm soluții, să încercăm să reinventăm roata, ca mai apoi să înțelegem un algoritm esențial ce a fost deja inventat, nu am putea spune că am programat.



7. BIBLIOGRAFIE

- [1] Wikipedia, *Single-page application*, disponibil la: https://en.wikipedia.org/wiki/Single-page_application .
- [2] Ganssle, J. G., „A Guide to Debouncing”, The Ganssle Group, University of Utah, Rev. 3, Iunie 2008.
- [3] Mozilla Developer Network (MDN), „Single-page application - Web terms”, 2024. Disponibil la: <https://developer.mozilla.org/en-US/docs/Glossary/SPA>.
- [4] Gamma, E., Helm, R., Johnson, R., & Vlissides, J., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional, 1994.
- [5] Fowler, M., *Patterns of Enterprise Application Architecture*, Addison-Wesley Professional, 2002.
- [6] Google Search Central, „Largest Contentful Paint (LCP)”, 2024. Disponibil la: <https://developers.google.com/search/docs/appearance/core-web-vitals>.
- [7] web.dev, „Interaction to Next Paint (INP)”, 2024, Disponibil la: <https://web.dev/articles/inp>.



8. ANEXE

8.1. Anexa 1 – Module de securitate

CartController.php – Decriptare & Validare

```
138 $encryptedDataWithTag = base64_decode($encryptedDataB64);
139 $iv = base64_decode($ivB64);
140
141
142 $tagLength = 16;
143 $ciphertextLength = strlen($encryptedDataWithTag) - $tagLength;
144 $ciphertext = substr($encryptedDataWithTag, 0, $ciphertextLength);
145 $tag = substr($encryptedDataWithTag, -$tagLength);
146
147
148 $jsonPayload = openssl_decrypt($ciphertext, 'aes-256-gcm', $decryptedAesKey, OPENSSL_RAW_DATA, $iv, $tag);
149
150 if ($jsonPayload === false) {
151     sendError("Data decryption failed.");
152 }
153
154 $paymentData = json_decode($jsonPayload, true);
155 if (!$paymentData) {
156     sendError("Invalid decrypted payload.");
157 }
158
159
160 $_POST['pickup_time'] = $paymentData['pickup_time'] ?? '';
161 $_POST['token'] = $paymentData['token'] ?? '';
162 $_POST['payment_method'] = 'card'; // logic hardcodet for this flow
163
164 $last4 = substr($paymentData['cardNumber'] ?? '0000', -4);
165 error_log("Payment Processed: Card ending in $last4");
166
167 $this->checkoutLegacy();
```



ReservationController.php – Suprapunerea Intervalelor

```
217 $stmt = $this->conn->prepare("
218     SELECT reservation_time FROM reservations
219     WHERE table_id = ?
220     AND status != 'deleted'
221     AND TIMESTAMPDIFF(MINUTE, reservation_time, ?) > -80
222     AND TIMESTAMPDIFF(MINUTE, reservation_time, ?) < 60
223 ");
224 $stmt->bind_param("iss", $tableId, $resTime, $resTime);
225 $stmt->execute();
226 $conflictRes = $stmt->get_result()->fetch_assoc();
227 $stmt->close();
```



menu.php – Algoritmul „Debounce”

```
280 const sortSelect = document.getElementById('sort-select');
281 if(sortSelect) {
282     sortSelect.addEventListener('change', (e) => {
283         const val = e.target.value;
284         const currentParams = new URLSearchParams(window.location.search);
285         currentParams.delete('page');
286         currentParams.set('sort', val);
287         fetchFilteredContent(currentParams);
288     });
289 }
290
291
292 const searchInput = document.getElementById('menuSearchInput');
293 const searchBtn = document.getElementById('search-btn');
294 let searchTimeout;
295
296 const performSearch = (val) => {
297     const currentParams = new URLSearchParams(window.location.search);
298     currentParams.delete('page');
299     if(val) currentParams.set('q', val);
300     else currentParams.delete('q');
301
302     fetchFilteredContent(currentParams);
303 };
304
305 const handleSearch = () => {
306     const val = searchInput.value.trim();
307     performSearch(val);
308 };
309
310 if(searchBtn) {
311     searchBtn.addEventListener('click', handleSearch);
312 }
313
314 if(searchInput) {
315     searchInput.addEventListener('input', (e) => {
316         const val = e.target.value.trim();
317         clearTimeout(searchTimeout);
318         searchTimeout = setTimeout(() => {
319             performSearch(val);
320         }, 300); // 300ms debounce
321     });
322 }
```



CartController.php – Algoritm Fiscal

```
496 while ($row = $result->fetch_assoc()) {
497     $id = $row['id'];
498     $qty = $_SESSION['cart'][$id];
499     $price = floatval($row['price']);
500     $discount = intval($row['discount'] ?? 0);
501     $tvaCode = strtoupper($row['tva_code'] ?? 'A');
502     $tvaRate = $tvaRates[$tvaCode] ?? 0;
503
504     $effectivePrice = $price;
505     if ($discount > 0) {
506         $effectivePrice = $price - ($price * $discount / 100);
507     }
508
509     $lineOriginal = $price * $qty;
510     $lineFinal = $effectivePrice * $qty;
511
512     $taxAmount = $lineFinal * $tvaRate / (100 + $tvaRate);
513
514     $tvaTotal += $taxAmount;
515
516     if (!isset($taxBreakdown[$tvaCode])) {
517         $taxBreakdown[$tvaCode] = ['rate' => $tvaRate, 'amount' => 0, 'net' => 0];
518     }
519     $taxBreakdown[$tvaCode]['amount'] += $taxAmount;
520     $taxBreakdown[$tvaCode]['net'] += ($lineFinal - $taxAmount);
521
522     $row['quantity'] = $qty;
523     $row['original_price'] = $price;
524     $row['effective_price'] = $effectivePrice;
525     $row['line_total'] = $lineFinal;
526     $row['tva_code'] = $tvaCode;
527
528     $items[] = $row;
529     $originalSubtotal += $lineOriginal;
530     $finalTotal += $lineFinal;
531 }
```



ui.js – Ripple Effect

```
152 document.addEventListener("click", function (e) {
153     const target = e.target.closest(selector);
154
155     if (target) {
156         if (!target.classList.contains("btn-ripple")) {
157             target.classList.add("btn-ripple");
158         }
159
160         const circle = document.createElement("span");
161         const diameter = Math.max(target.clientWidth, target.clientHeight);
162         const radius = diameter / 2;
163
164         const rect = target.getBoundingClientRect();
165
166         circle.style.width = circle.style.height = `${diameter}px`;
167         circle.style.left = `${e.clientX - rect.left - radius}px`;
168         circle.style.top = `${e.clientY - rect.top - radius}px`;
169         circle.classList.add("ripple");
170
171         const ripple = target.getElementsByClassName("ripple")[0];
172         if (ripple) {
173             ripple.remove();
174         }
175
176         target.appendChild(circle);
177     }
178 });
179 }
```



checkoutEncrypted()

```
96 private function checkoutEncrypted() {
97     ini_set('display_errors', '0');
98     error_reporting(E_ALL);
99
100     $encryptedKeyB64 = $_POST['encrypted_key'] ?? '';
101     $encryptedDataB64 = $_POST['encrypted_data'] ?? '';
102     $ivB64 = $_POST['iv'] ?? '';
103
104     if (empty($encryptedKeyB64) || empty($encryptedDataB64) || empty($ivB64)) {
105         if (isset($_POST['payment_method']) && !isset($_POST['encrypted_key'])) {
106             $this->checkoutLegacy();
107             return;
108         }
109         sendError("Encryption data missing.");
110     }
111     if (empty($_SESSION['rsa_private_key'])) {
112         sendError("Session expired (Key missing). Please refresh.");
113     }
114     $encryptedKey = base64_decode($encryptedKeyB64);
115     $privateKey = openssl_pkey_get_private($_SESSION['rsa_private_key']);
116     $decryptedAesKey = null;
117
118     if (!openssl_private_decrypt($encryptedKey, $decryptedAesKey, $privateKey, OPENSSL_PKCS1_OAEP_PADDING)) {
119         $err = openssl_error_string();
120         file_put_contents(__DIR__ . '/../ssl_debug.log', "Decryption Failed: $err\nkey len: " . strlen($encryptedKey) . "\n", FILE_APPEND);
121         sendError("Secure handshake failed. Please retry.");
122     }
123
124     $encryptedDataWithTag = base64_decode($encryptedDataB64);
125     $iv = base64_decode($ivB64);
126
127     $tagLength = 16;
128     $ciphertextLength = strlen($encryptedDataWithTag) - $tagLength;
129     $ciphertext = substr($encryptedDataWithTag, 0, $ciphertextLength);
130     $tag = substr($encryptedDataWithTag, -$tagLength);
131
132     $jsonPayload = openssl_decrypt($ciphertext, 'aes-256-gcm', $decryptedAesKey, OPENSSL_RAW_DATA, $iv, $tag);
133
134     if ($jsonPayload === false) {
135         sendError("Data decryption failed.");
136     }
137
138     $paymentData = json_decode($jsonPayload, true);
139     if (!$paymentData) {
140         sendError("Invalid decrypted payload.");
141     }
142
143     $_POST['pickup_time'] = $paymentData['pickup_time'] ?? '';
144     $_POST['token'] = $paymentData['token'] ?? '';
145     $_POST['payment_method'] = 'card';
146
147     $last4 = substr($paymentData['cardNumber'] ?? '0000', -4);
148     error_log("Payment Processed: Card ending in $last4");
149
150     $this->checkoutLegacy();
151 }
```



8.2. Anexa 2 – Motorul de navigare SPA și afișare

router.js – Algoritmul „Hydration” & SPA Routing

```
26 const navbar = document.querySelector('.header');
27 if (page === 'admin') {
28   if (navbar) navbar.style.display = 'none';
29 } else {
30   if (navbar) navbar.style.display = 'block';
31 }
32
33 const routes = window.APP_CONFIG?.routes || {};
34 const url = routes[page];
35
36 if (!url) {
37   app.innerHTML = "<h2>404 Page not found</h2>";
38   updateHero(page);
39   setActiveLink(page);
40   throw new Error("Page not found");
41 }
42
43 if (page === "admin") {
44   const roles = window.APP_CONFIG?.currentUserRoles || [];
45   if (!roles.includes("admin") && !roles.includes("employer")) {
46     showModal("Nu esti autorizat să accesezi pagina de administrare.");
47     setActiveLink("home");
48     return loadPage("home");
49   }
50 }
51
52 const fetchUrl = `${url}?t=${new Date().getTime()}`;
53 app.classList.add('page-transition-exit');
54
55 let isFetched = false;
56 const fetchPromise = safeFetch(fetchUrl)
57   .then(res => res.text())
58   .then(html => {
59     isFetched = true;
60     return html;
61   })
62   .catch(err => {
63     console.error("Failed to load page:", err);
64     isFetched = true;
65     return "<h2>Error loading page</h2>";
66   });
```



updateCoordinates()

```
92 private function updateCoordinates() {
93     $id = intval($_POST['id'] ?? 0);
94     $x = floatval($_POST['x'] ?? 0);
95     $y = floatval($_POST['y'] ?? 0);
96     $w = floatval($_POST['width'] ?? 5);
97     $h = floatval($_POST['height'] ?? 5);
98
99     if ($id <= 0) sendError("Invalid ID");
100
101     // Clamp values (Percentages)
102     $x = max(0, min(100, $x));
103     $y = max(0, min(100, $y));
104     $w = max(1, min(100, $w)); // Min 1% width
105     $h = max(1, min(100, $h)); // Min 1% height
106
107     $stmt = $this->conn->prepare("UPDATE tables SET x_pos=?, y_pos=?, width=?, height=? WHERE ID=?");
108     $stmt->bind_param("ddddd", $x, $y, $w, $h, $id);
109
110     if($stmt->execute()) {
111         sendSuccess(['message' => 'Coordinates and size updated']);
112     } else {
113         sendError("Failed to update status");
114     }
115 }
```

getAll()

```
34 private function getAll() {
35     $search = trim($_POST['search'] ?? $_GET['search'] ?? '');
36
37     $sql = "SELECT id, username, email, role, PuncteFidelitate, PPicture, is_blacklisted FROM users";
38
39     if (!empty($search)) {
40         // Secure search
41         $search = "%" . $this->conn->real_escape_string($search) . "%";
42         $sql .= " WHERE username LIKE '$search' OR email LIKE '$search'";
43     }
44
45     $sql .= " ORDER BY id DESC LIMIT 50";
46
47     $res = $this->conn->query($sql);
48     $users = [];
49     while($row = $res->fetch_assoc()) {
50         $users[] = $row;
51     }
52     sendSuccess(['data' => $users]);
53 }
```



showContentModal()

```
168 export function showContentModal(htmlContent) {
169   let modal = document.getElementById("content-modal");
170
171   if (!modal) {
172     modal = document.createElement("div");
173     modal.id = "content-modal";
174
175     // Create base structure
176     modal.innerHTML = `
177     <div class="modal-card" style="position: relative;">
178       <button class="close-btn" style="position: absolute; top: 10px; right: 10px; background: none; border: none; font-size: 20px; cursor: pointer; color: #555;">&times;</button>
179       <div id="modal-content-body"></div>
180     </div>
181     `;
182
183     // Inject content
184     const body = modal.querySelector("#modal-content-body");
185     body.innerHTML = htmlContent;
186
187     // Show
188     modal.style.display = "flex";
189     const card = modal.querySelector(".modal-card");
190     requestAnimationFrame(() => {
191       card.style.opacity = "1";
192       card.style.transform = "scale(1)";
193     });
194   }
195 }
```



8.3. Anexa 3 – Module de design CSS3

Slider

```
123 .hero-slider .slide {
124     position: absolute;
125     inset: 0;
126     width: 100%;
127     height: 100%;
128     object-fit: contain;
129
130     opacity: 0;
131     transform: scale(1.05) translateY(8px);
132     transition:
133         opacity 1s ease,
134         transform 1s ease;
135 }
136
137 /* visible image */
138 .hero-slider .slide.active {
139     opacity: 1;
140     transform: scale(1) translateY(0);
141 }
142
143 /* Slider Animations */
144 .hero-slider .slide {
145     position: absolute;
146     inset: 0;
147     width: 100%;
148     height: 100%;
149     margin-top: 40px;
150     object-fit: cover;
151     opacity: 0;
152     transform: scale(1.05);
153     /* Slight zoom start */
154     transition: opacity 1s cubic-bezier(0.4, 0, 0.2, 1), transform 1s cubic-bezier(0.4, 0, 0.2, 1);
155     z-index: 1;
156     border-radius: 20px;
157     box-shadow: 0 4px 15px rgba(129, 104, 66, 0.3);
158     border: 2px solid rgb(41, 27, 6);
159 }
160
161 .hero-slider .slide.active {
162     opacity: 1;
163     transform: scale(1);
164     /* Zoom to normal */
165     z-index: 2;
166 }
167
168 #slide-content-wrapper {
169     transition: opacity 0.3s ease, transform 0.3s ease;
170 }
171
172 /* Ensure controls are above slides */
173 .slider-controls {
174     z-index: 10;
175     position: relative;
176 }
```



Card

```
110 .card-perspective { perspective: 1000px; width: 100%; height: 200px; margin-bottom: 40px; }
111 .card-inner { position: relative; width: 100%; height: 100%; transition: transform 0.6s cubic-bezier(0.4, 0, 0.2, 1); transform-style: preserve-3d; }
112 .card-perspective.flipped .card-inner { transform: rotate(180deg); }
113
114 .card-face {
115   position: absolute; inset: 0; backface-visibility: hidden;
116   border-radius: 10px; padding: 20px;
117   background: linear-gradient(135deg, #2c3e50 49%, #4a5568 49%, #4a5568 100%); /* Coffee gradient */
118   color: white; display: flex; flex-direction: column; justify-content: space-between;
119   box-shadow: 0 20px 40px -10px rgba(4, 24, 16, 0.5);
120 }
121
122 .card-face:after {
123   content: ''; position: absolute; inset: 0;
124   background: linear-gradient(to right, transparent 49%, #2c3e50 49%, #2c3e50 51%, transparent 51%);
125   opacity: 0.5; pointer-events: none;
126 }
127
128 .card-back { transform: rotate(180deg); padding: 0; justify-content: flex-start; }
129
130 /* Branding Colors */
131 .visa .card-face { background: linear-gradient(135deg, #662d91, #004d40); }
132 .mastercard .card-face { background: linear-gradient(135deg, #e61e24, #d9534f); }
133 .amex .card-face { background: linear-gradient(135deg, #006d3c, #004d40); }
134 .discover .card-face { background: linear-gradient(135deg, #f7931e, #f7931e); }
135 .diners .card-face { background: linear-gradient(135deg, #2c3e50, #2c3e50); }
136 .jcb .card-face { background: linear-gradient(135deg, #000000, #000000); }
137
138 /* Card Text Styling */
139 .card-title { background: linear-gradient(135deg, #f7931e, #f7931e); border-radius: 5px; width: 40px; height: 32px; }
140 .card-number-display { font-family: 'Courier New', monospace; font-size: 1.6em; letter-spacing: 0.5em; text-shadow: 0 2px 2px rgba(0, 0, 0, 0.3); margin-top: 10px; }
141 .card-label { font-size: 0.8em; text-transform: uppercase; color: rgba(255, 255, 255, 0.7); letter-spacing: 0.5em; margin-bottom: 2px; }
142 .card-val { font-size: 1.2em; font-family: 'Courier New', monospace; text-shadow: 0 2px 2px rgba(0, 0, 0, 0.3); }
```



Floor Plan

```
1017 async function loadTables() {
1018     const res = await apiRequest("table", "get_all");
1019     if (res.success) {
1020         document.getElementById("table-count-display").innerText = res.data.length;
1021         renderFloorPlan(res.data, res.background);
1022         renderTableList(res.data);
1023     }
1024 }
1025
1026 function renderTableList(tables) {
1027     const grid = document.getElementById("tables-grid");
1028     if (!grid) return;
1029     grid.innerHTML = tables
1030         .map(
1031             (t) => `
1032                 <div class="table-card status-${t.Status.toLowerCase().replace(
1033                     " ",
1034                     "-"
1035                 )}">
1036                     <div class="table-id">Table ${t.ID}</div>
1037                     <select class="table-status-select" onchange="updateTableStatus(${t.ID
1038                         }, this.value)">
1039                         <option value="Libera" ${t.Status == "Libera" ? "selected" : ""
1040                             }>Libera</option>
1041                         <option value="Ocupata" ${t.Status == "Ocupata" ? "selected" : ""
1042                             }>Ocupata</option>
1043                         <option value="Rezervata" ${t.Status == "Rezervata" ? "selected" : ""
1044                             }>Rezervata</option>
1045                         <option value="Inactiva" ${t.Status == "Inactiva" ? "selected" : ""
1046                             }>Inactiva</option>
1047                     </select>
1048                     <div style="margin-top:10px;">
1049                         <button class="btn btn-sm btn-edit" onclick="openTableProps(${t.ID
1050                             }, '${t.shape}', ${t.width}, ${t.height
1051                                 })"><i class="fas fa-cog"></i></button>
1052                     </div>
1053                 </div>
1054             `
1055         )
1056         .join("");
1057 }
```



8.4. Anexa 4 – Tabele din baza de date

Tabelul users (Placeholder1) (Placeholder1)

```
401 CREATE TABLE `users` (  
402   `id` int NOT NULL AUTO_INCREMENT,  
403   `email` varchar(255) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,  
404   `username` varchar(50) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci DEFAULT NULL,  
405   `password` varchar(255) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci DEFAULT NULL,  
406   `role` enum('user','admin','employer') CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci DEFAULT 'user',  
407   `created_at` timestamp NULL DEFAULT CURRENT_TIMESTAMP,  
408   `PuncteFidelitate` int DEFAULT NULL,  
409   `PPicture` varchar(2048) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci DEFAULT 'assets/public/default.png',  
410   `oauth_provider` varchar(50) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci DEFAULT NULL,  
411   `oauth_uid` varchar(255) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci DEFAULT NULL,  
412   `is_blacklisted` tinyint(1) DEFAULT '0',  
413   `blacklist_reason` text CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci,  
414   PRIMARY KEY (`id`),  
415   UNIQUE KEY `email` (`email`)  
416 ) ENGINE=InnoDB AUTO_INCREMENT=6 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Tabelul products

```
193 CREATE TABLE `products` (  
194   `id` int NOT NULL AUTO_INCREMENT,  
195   `name` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,  
196   `description` text CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci,  
197   `tags` text COLLATE utf8mb4_unicode_ci,  
198   `ingredients` text CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci,  
199   `quantity` varchar(50) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci DEFAULT NULL,  
200   `price` decimal(10,2) NOT NULL,  
201   `image_path` varchar(255) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci DEFAULT NULL,  
202   `category` enum('coffee','tea','chocolate','refreshment','signature','addon') CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci DEFAULT 'coffee',  
203   `created_at` timestamp NULL DEFAULT CURRENT_TIMESTAMP,  
204   `discount` int DEFAULT '0',  
205   `tva_code` char(1) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci DEFAULT 'A',  
206   PRIMARY KEY (`id`)  
207 ) ENGINE=InnoDB AUTO_INCREMENT=26 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Tabelul orders

```
135 CREATE TABLE `orders` (  
136   `id` int NOT NULL AUTO_INCREMENT,  
137   `user_id` int NOT NULL,  
138   `pickup_time` datetime NOT NULL,  
139   `status` enum('pending','preparing','ready','completed','cancelled') CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci DEFAULT 'pending',  
140   `payment_method` varchar(50) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci DEFAULT 'card',  
141   `total_price` decimal(10,2) NOT NULL,  
142   `created_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,  
143   `table_id` int DEFAULT NULL,  
144   `completed_at` datetime DEFAULT NULL,  
145   PRIMARY KEY (`id`),  
146   KEY `user_id` (`user_id`),  
147   KEY `table_id` (`table_id`),  
148   CONSTRAINT `orders_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE  
149 ) ENGINE=InnoDB AUTO_INCREMENT=43 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```